

UNIVERSITÉ HASSAN II DE CASABLANCA

Faculté des Sciences Ben M'Sick • Département Mathématiques et Informatique

Semestre 6 • Module : Cybersécurité des Systèmes d'Information

RAPPORT DE PROJET

Sécurisation des API REST avec JWT et Spring Security

Projet : SecureAPI — Plateforme de gestion sécurisée des utilisateurs

ENCADRÉ PAR

Pr. Hafsa BENADDI

RÉALISÉ PAR

Chaimaa EL MEHDAOUI
Walid Ait Sidi Ahmed

Stack technique :

Spring Boot 3.2 • Spring Security 6 • JWT 0.12
React 18 • MySQL 8 • BCrypt (facteur 12)

Année Universitaire 2025 — 2026

Table des matières

1	Introduction	2
2	Contexte, Problématique et Objectifs	2
2.1	Contexte	2
2.2	Problématique	2
2.3	Objectifs	2
3	Technologies et Outils Utilisés	3
4	Architecture du Projet	3
4.1	Architecture générale	3
4.2	Flux d'authentification JWT	3
5	Implémentation	3
5.1	Backend — Spring Boot & Spring Security	3
5.1.1	Configuration de la sécurité (SecurityConfig.java)	3
5.1.2	Filtre JWT (JwtAuthFilter.java)	4
5.1.3	Hachage BCrypt	4
5.2	Endpoints de l'API REST	4
5.3	Frontend — React 18	4
5.4	Base de données	5
6	Guide de Démarrage	5
6.1	Prérequis	5
6.2	Étape 1 — Initialisation de la base de données	5
6.3	Étape 2 — Configuration et lancement du backend	5
6.4	Étape 3 — Lancement du frontend React	6
6.5	Récapitulatif des URLs	6
7	Résultats — Présentation de l'Application	6
7.1	Page d'accueil	6
7.2	Authentification : Connexion et Inscription	7
7.3	Tableau de bord et visualiseur JWT	7
7.4	Profil utilisateur	8
8	Sécurité : Mécanismes Implémentés	9
9	Conclusion	9

1 Introduction

La multiplication des architectures distribuées et des services mobiles a fait des **API REST** (Representational State Transfer) le standard incontournable des échanges entre systèmes. Cependant, l'exposition de ces interfaces sur des réseaux publics soulève des enjeux de sécurité fondamentaux : comment authentifier les utilisateurs sans maintenir d'état côté serveur ? Comment protéger les ressources selon les privilèges de chaque acteur ?

Ce projet répond à ces questions en réalisant **SecureAPI**, une application full-stack complète qui illustre les mécanismes de sécurisation modernes : authentification JWT stateless, contrôle d'accès basé sur les rôles (RBAC), hachage BCrypt et protection des routes côté client.

2 Contexte, Problématique et Objectifs

2.1. Contexte

Dans le cadre du module *Cybersécurité des Systèmes d'Information*, ce projet applique les principes étudiés en cours à un cas réel et fonctionnel. L'application SecureAPI simule une plateforme de gestion d'utilisateurs à accès différencié (utilisateurs standard et administrateurs), exposée via une API REST sécurisée.

2.2. Problématique

- **Authentification sans état** : les sessions HTTP classiques ne conviennent pas aux API REST distribuées et sans état (*stateless*).
- **Intégrité des données transmises** : comment garantir qu'un token d'identité n'a pas été falsifié ?
- **Contrôle d'accès granulaire** : certaines ressources doivent être réservées aux administrateurs, d'autres aux utilisateurs simples.
- **Stockage sécurisé des mots de passe** : un vol de base de données ne doit pas exposer les mots de passe en clair.

2.3. Objectifs

- Implémenter JWT (RFC 7519) avec HMAC-SHA256 12)
- Configurer Spring Security 6 en mode stateless
- Mettre en place le RBAC : ROLE_USER / ROLE_ADMIN 3
- Hacher les mots de passe avec BCrypt (coût
- Développer un frontend React avec routes protégées
- Documenter l'API via Swagger / OpenAPI 3
- Tester tous les endpoints avec Postman

3 Technologies et Outils Utilisés

Couche	Technologie	Rôle
Backend	Spring Boot 3.2	Framework applicatif Java
Backend	Spring Security 6	Filtrage, autorisation, gestion des rôles
Backend	JJWT 0.12.5	Génération et validation des tokens JWT
Backend	Spring Data JPA	Persistance ORM vers MySQL
Backend	BCrypt (coût 12)	Hachage sécurisé des mots de passe
Backend	SpringDoc OpenAPI 2.3	Documentation Swagger UI interactive
Frontend	React 18 + Vite 5	Interface utilisateur SPA
Frontend	React Router v6	Routage côté client et routes protégées
Frontend	Axios 1.x	Client HTTP avec intercepteurs JWT
Base de données	MySQL 8	Stockage relationnel des utilisateurs et rôles
Tests	Postman	Validation des endpoints et scénarios d'erreur
Build	Maven 3.8	Gestion des dépendances backend

Table 1. Stack technique complète du projet SecureAPI

4 Architecture du Projet

4.1. Architecture générale

L'application suit une architecture **3-tiers** : client React (présentation), API REST Spring Boot (logique métier et sécurité), et base de données MySQL (persistance). Toute communication entre le frontend et le backend s'effectue en JSON sur HTTP avec authentification par token JWT.

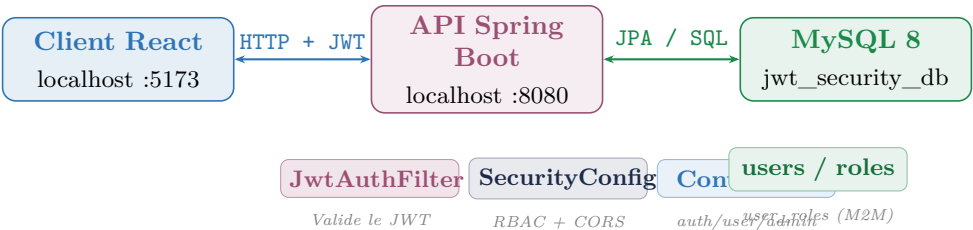


Figure 1. Architecture 3-tiers de l'application SecureAPI

4.2. Flux d'authentification JWT

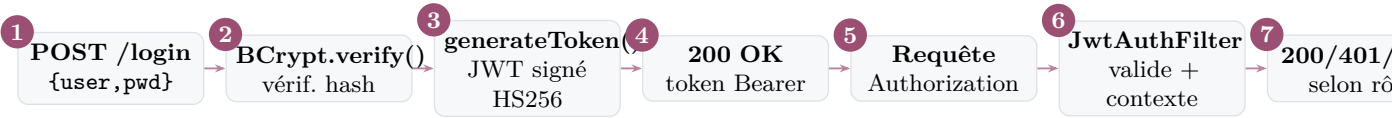


Figure 2. Flux complet d'authentification JWT — de la connexion à l'accès aux ressources

5 Implémentation

5.1. Backend — Spring Boot & Spring Security

5.1.1. Configuration de la sécurité (SecurityConfig.java)

La classe `SecurityConfig` constitue le cœur de la sécurité. Elle définit la politique **STATELESS** (aucune session HTTP), les règles d'accès par route et par rôle, et insère le filtre JWT avant le filtre d'authentification standard de Spring.

```

http
    .csrf(disabled) // API REST stateless
    .sessionManagement(STATELESS) // aucune session serveur
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/auth/**").permitAll()
        .requestMatchers("/api/user/**").hasAnyRole("USER", "ADMIN")
        .requestMatchers("/api/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()
    )
    .exceptionHandling(ex -> ex
        .authenticationEntryPoint(/* retourne JSON 401 */)
        .accessDeniedHandler(/* retourne JSON 403 */)
    )
    .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthFilter.class);

```

5.1.2. Filtre JWT (*JwtAuthFilter.java*)

Ce filtre hérite de `OncePerRequestFilter` : il s'exécute exactement une fois par requête, extrait le token de l'en-tête `Authorization: Bearer <token>`, valide la signature HMAC-SHA256 et la date d'expiration, puis injecte l'objet `Authentication` dans le `SecurityContextHolder` pour les vérifications de rôles suivantes.

5.1.3. Hachage BCrypt

BCrypt génère un sel aléatoire unique à chaque hachage, rendant inutiles les tables arc-en-ciel. Le facteur de coût 12 correspond à $2^{12} = 4096$ itérations, rendant les attaques par force brute prohibitivement coûteuses.

5.2. Endpoints de l'API REST

Méthode	Route	Description	Accès
POST	/api/auth/register	Inscription d'un nouvel utilisateur	Public
POST	/api/auth/login	Connexion et génération du JWT	Public
GET	/api/user/profile	Profil de l'utilisateur connecté	USER
GET	/api/admin/users	Liste de tous les utilisateurs	ADMIN
DELETE	/api/admin/users/{id}	Suppression d'un utilisateur	ADMIN
GET	/swagger-ui.html	Documentation OpenAPI interactive	Public

Table 2. Endpoints de l'API REST SecureAPI avec niveaux d'accès

5.3. Frontend — React 18

Le frontend est développé avec React 18 et React Router v6. L'état d'authentification global est géré via `AuthContext` (React Context API). Le composant `ProtectedRoute` agit comme un Higher-Order Component : il redirige vers `/login` si l'utilisateur n'est pas connecté, ou vers `/access-denied` si son rôle est insuffisant. L'instance `Axios` injecte automatiquement le token JWT dans chaque requête sortante et détecte les erreurs 401 pour rediriger vers la page de connexion.

5.4. Base de données

Table	Colonnes principales
users	id, username (UNIQUE), email (UNIQUE), password (BCrypt), first_name, last_name, enabled, created_at
roles	id, role_name (UNIQUE) : ROLE_USER, ROLE_ADMIN
user_roles	user_id (FK), role_id (FK) — relation Many-to-Many

Table 3. Schéma de la base de données MySQL

6 Guide de Démarrage

6.1. Prérequis

Environnement nécessaire

- **Java** 17 ou supérieur (JDK)
- **Maven** 3.8 ou supérieur
- **MySQL** 8.0 ou supérieur
- **Node.js** 18 ou supérieur
- **npm** 9 ou supérieur
- Navigateur web moderne

Vérification rapide des versions installées :

```
java -version      # >= 17.0.x
mvn -version      # >= 3.8.x
mysql --version    # >= 8.0.x
node -v           # >= 18.x.x
npm -v            # >= 9.x.x
```

6.2. Étape 1 — Initialisation de la base de données

```
mysql -u root -p

CREATE DATABASE jwt_security_db CHARACTER SET utf8mb4;
SOURCE /chemin/vers/database/schema.sql;
```

Les tables `users`, `roles` et `user_roles` sont créées automatiquement. Un compte administrateur est provisionné au premier démarrage par `DataInitializer.java` :

Compte administrateur par défaut

Username : admin • **Mot de passe** : Admin@123 • **Rôle** : ROLE_ADMIN

6.3. Étape 2 — Configuration et lancement du backend

Ouvrir `backend/src/main/resources/application.properties` et renseigner les identifiants MySQL :

```
spring.datasource.username=root
spring.datasource.password=VOTRE_MOT_DE_PASSE
```

Puis compiler et lancer le serveur :

```
cd backend
mvn clean install -DskipTests
mvn spring-boot:run
# API disponible sur http://localhost:8080
# Swagger UI sur http://localhost:8080/swagger-ui.html
```

6.4. Étape 3 — Lancement du frontend React

```
cd frontend
npm install
npm run dev
# Application sur http://localhost:5173
```

6.5. Récapitulatif des URLs

Service	URL	Description
Frontend	http://localhost:5173	Interface React (page d'accueil)
API Backend	http://localhost:8080	Serveur Spring Boot
Swagger UI	http://localhost:8080/swagger-ui.html	Documentation interactive
OpenAPI JSON	http://localhost:8080/api-docs	Spécification OpenAPI 3

Table 4. URLs d'accès à l'application

7 Résultats — Présentation de l'Application

7.1. Page d'accueil

La page d'accueil de SecureAPI présente la plateforme avec une interface sombre et moderne. Elle met en avant les fonctionnalités clés (JWT Stateless, BCrypt, RBAC, OpenAPI) et propose deux actions principales : se connecter ou créer un compte.

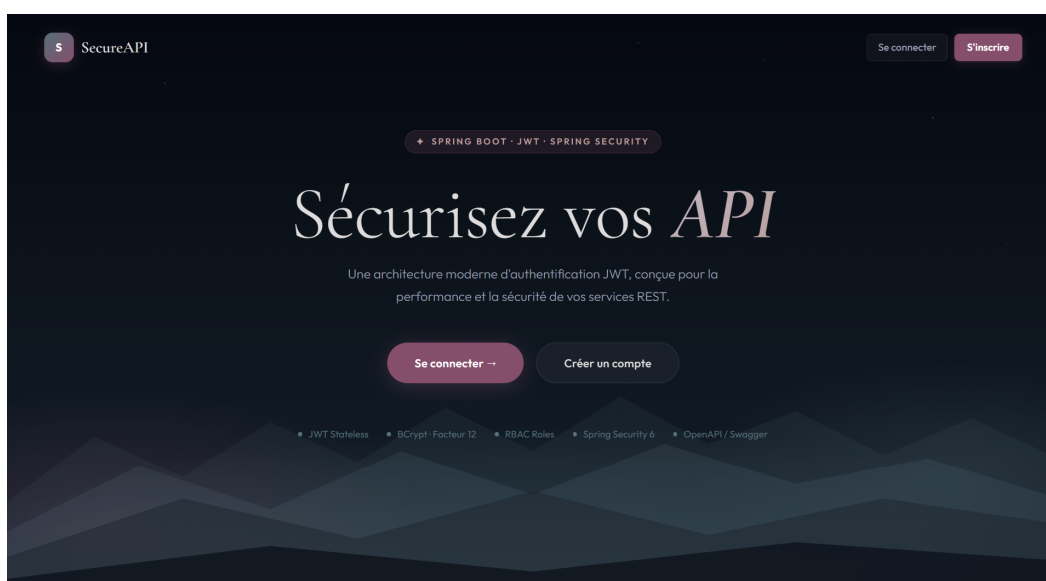
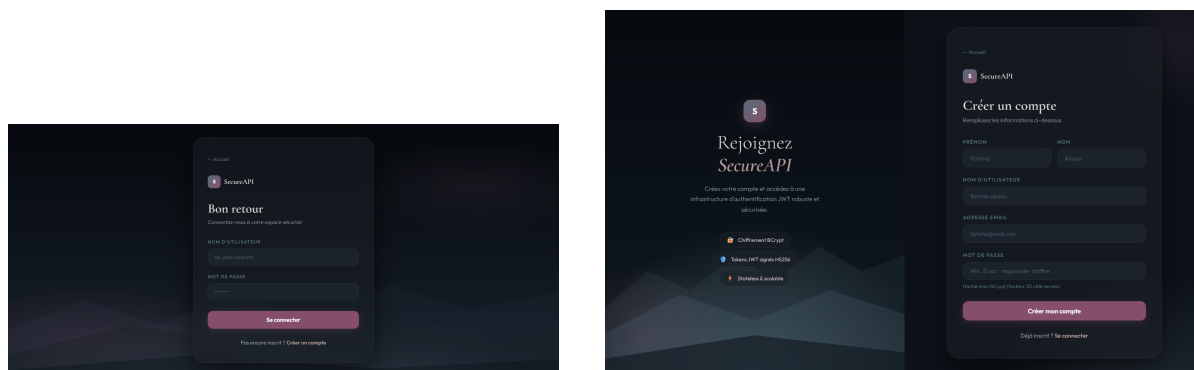


Figure 3. Page d'accueil — SecureAPI avec présentation des mécanismes de sécurité

7.2. Authentification : Connexion et Inscription



(a) Page de connexion — saisie des identifiants **(b)** Page d'inscription — formulaire sécurisé BCrypt

Figure 4. Interfaces d'authentification de SecureAPI

La page de connexion envoie les identifiants en `POST /api/auth/login`. En cas de succès, le token JWT est stocké dans le `localStorage` et l'utilisateur est redirigé vers le tableau de bord. Le mot de passe est immédiatement haché avec BCrypt côté serveur avant tout stockage.

7.3. Tableau de bord et visualiseur JWT

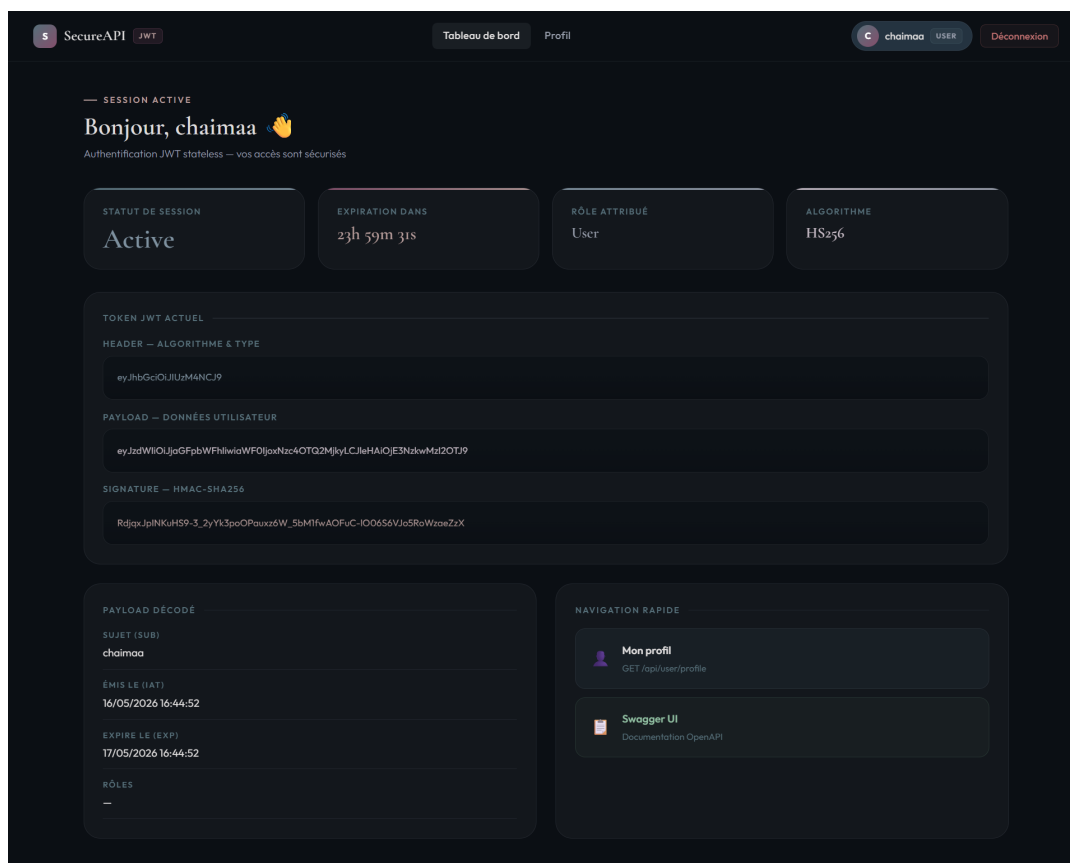


Figure 5. *Tableau de bord — décomposition en temps réel du token JWT (Header, Payload, Signature)*

Le tableau de bord constitue la pièce centrale pédagogique du projet. Il affiche :

- Le **statut de session** et le **compte à rebours d'expiration** du token (24h) ainsi que l'algorithme utilisé (HS256)

- La **décomposition visuelle du JWT** en trois parties : Header (algorithme + type), Payload (sub, iat, exp, roles) et Signature HMAC-SHA256
- Le **payload décodé** en JSON lisible sans bibliothèque externe (via `atob()` natif du navigateur)

7.4. Profil utilisateur

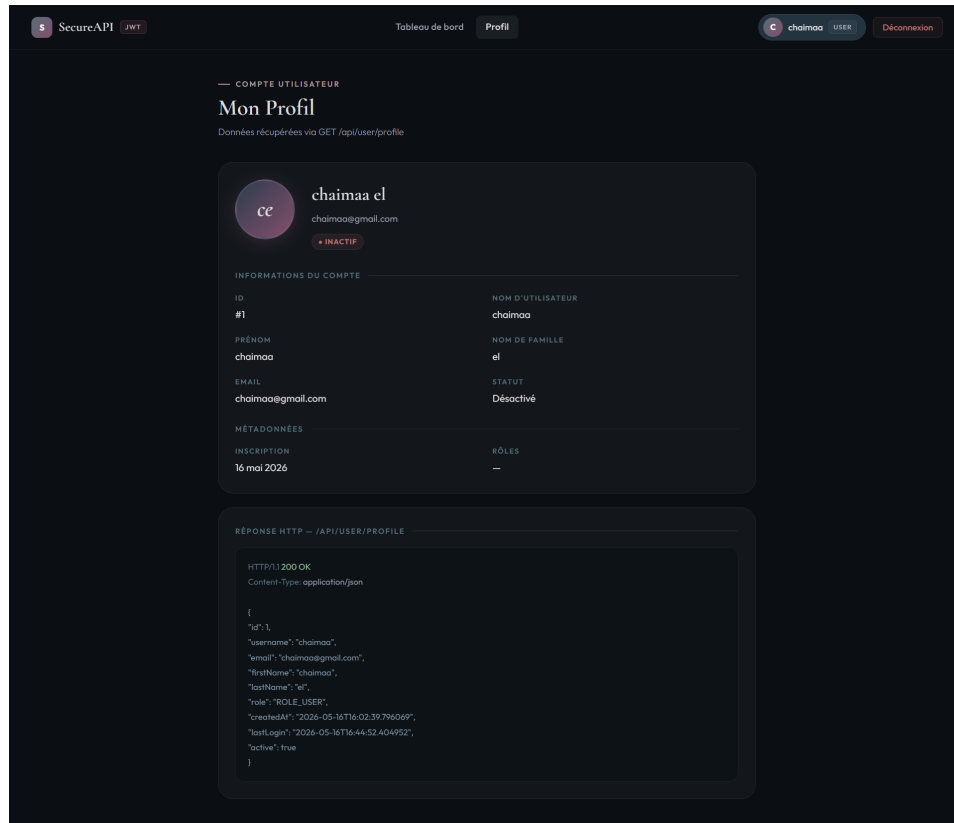


Figure 6. Page profil — données récupérées via `GET /api/user/profile` avec le token JWT

La page profil illustre un appel à un **endpoint protégé** : Axios injecte automatiquement le token dans l'en-tête `Authorization`, Spring Security valide ce token via `JwtAuthFilter`, et la réponse JSON contient les données de l'utilisateur authentifié, sans le mot de passe.

8 Sécurité : Mécanismes Implémentés

Menace	Mécanisme de protection	Implémentation
Token falsifié	Signature HMAC-SHA256	Clé secrète 256 bits (Base64)
Token expiré	Vérification de <code>exp</code>	<code>JwtUtil.isTokenExpired()</code>
Mot de passe volé	BCrypt adaptatif	<code>BCryptPasswordEncoder(12)</code>
Tables arc-en-ciel	Sel aléatoire BCrypt	Automatique à chaque hachage
Accès non autorisé	RBAC + filtre JWT	<code>@PreAuthorize</code> + <code>SecurityFilterChain</code>
Session hijacking	Stateless (pas de cookies)	<code>SessionCreationPolicy.STATELESS</code>
CORS non contrôlé	Liste d'origines explicite	<code>localhost:5173</code> uniquement
Données invalides	Validation <code>@Valid</code>	<code>@NotBlank</code> , <code>@Email</code> , <code>@Pattern</code>

Table 5. Tableau des menaces et contre-mesures implémentées dans SecureAPI

9 Conclusion

Ce projet a permis de concevoir et de déployer une solution complète de sécurisation d'API REST, démontrant l'articulation entre Spring Security 6, les JSON Web Tokens et les bonnes pratiques de cryptographie applicative (BCrypt). L'application SecureAPI valide les concepts clés du module : authentification sans état, contrôle d'accès basé sur les rôles, protection des routes côté frontend et côté backend, avec une interface pédagogique illustrant en temps réel la structure d'un token JWT.

Plusieurs perspectives d'amélioration sont envisageables : l'implémentation d'un mécanisme de *refresh token*, la mise en place d'un audit trail des connexions, l'ajout d'une authentification multi-facteurs (2FA/TOTP), et le déploiement en production avec HTTPS/TLS pour le chiffrement des communications.